Проектная траектория_ООП
Файл Правка Вид Инструменты Справка
100% Сохранить копию на Диске Редактировать копию
Поделиться Войти

ПРОЕКТНАЯ ТРАЕКТОРИЯ

по дисциплине «Объектно-ориентированное программирование. Современные паттерны»

Формула траектории: ИИ + оркестрация + архитектура + фиксация → работающий продукт и осознанная защита.

Методические указания для студентов

Москва · 2026

1. Назначение проектной траектории

Студент может выбрать проектную траекторию вместо обычного набора лабораторных работ. Такая замена не уменьшает объём обучения: она объединяет темы курса в разработке одного многомодульного приложения на C++ (или другом современном языке программирования по согласованию с преподавателем) и требует регулярного предъявления промежуточных результатов (4 отчетные даты в семестре).
Цель — научиться проектировать объектную модель, управлять разработкой с помощью ИИ, пользоваться Git, фиксировать работающие версии и доводить решение до состояния минимально законченного продукта (MVP).

Главный принцип: ИИ может предлагать и писать код, но студент отвечает за архитектуру, проверку, интеграцию и понимает большинство строк кода итогового проекта.

2. Расшифровка формулы

Компонент	Практический смысл
ИИ	Нейросеть используется как инженерный помощник: для обсуждения требований, декомпозиции, вариантов архитектуры, отдельных фрагментов реализации, тестов, поиска ошибок и документации. Встраивать нейросеть в сам продукт необязательно, если это отдельно не предусмотрено темой.
Оркестрация	Студент управляет цепочкой работ: формулирует подзадачи, передаёт контекст, проверяет ответы ИИ, объединяет модули, запускает сборку и тесты, исправляет ошибки и принимает решения. Оркестрация не означает обязательное использование LangChain, агентов или нескольких моделей.
Архитектура	Проект разделён на модули и классы с понятными обязанностями. Бизнес-логика отделена от пользовательского интерфейса и хранения данных. Наследование, полиморфизм, шаблоны и паттерны применяются только там, где решают реальную задачу.
Фиксация	Работа ведётся с Git (допускается использование локального Git, например, как в Xcode). Рабочие состояния сохраняются осмысленными коммитами, контрольные версии отмечаются тегами, изменения отражаются в CHANGELOG.md, существенные решения — в docs/DECISIONS.md.

3. Обязательный результат

К концу семестра студент представляет не набор разрозненных исходников, а репозиторий законченного учебного продукта, который можно собрать, запустить, протестировать и продемонстрировать:
рабочий исходный код (преимущественно на C++ или на другом языке программирования по согласованию с преподавателем);
система сборки CMake (если C++);
понятная модульная структура;
устойчивый основной пользовательский сценарий;

сохранение данных между запусками, если оно необходимо предметной области;
автоматические тесты ключевой бизнес-логики;
README с инструкциями по сборке и запуску;
краткое описание продукта, пользователя и возможной модели продажи;
история разработки в Git и журнал существенных эпизодов работы с ИИ.

4. Минимальные технические требования

Область	Минимум
Язык	C++11 или новее. Другие языки допустимы, но должны быть согласованы.
Объектная модель	Ориентир — не менее 4-6 содержательных классов. Искусственные классы ради количества не учитываются.
Модули	Не менее 3 логических модулей: например, domain, application, infrastructure/UI.
Инкапсуляция	Состояние объектов защищено; публичный интерфейс минимален и осмыслен.
Композиция	Используется как основной способ построения сложных объектов.
Полиморфизм	Не менее одного оправданного примера через интерфейс/абстрактный класс или шаблонную стратегию.
Ошибки	Некорректный ввод и ожидаемые сбои обрабатываются; программа не завершается аварийно в обычных сценариях.
Тесты	Проверяются ключевые правила предметной области и минимум один пограничный сценарий.
Интерфейс	Допустим консольный интерфейс (нужно согласование). GUI является расширением, а не заменой архитектуры.

5. Рекомендуемая архитектура

Ниже приведён ориентир, а не обязательный шаблон. Студент может предложить другую структуру и обосновать её.

Слой/модуль	Ответственность
Domain	Сущности, значения, правила предметной области. Не зависит от UI и конкретной базы данных.
Application	Пользовательские сценарии: создать заказ, выполнить сканирование, сформировать отчёт.
Infrastructure	Файлы, JSON/SQLite, сеть, системные API, логирование.
Presentation	CLI, Qt или другой интерфейс. Передаёт команды приложению и показывает результат.
Tests	Проверка доменной и прикладной логики независимо от интерфейса.

Архитектурная проверка: если замена консольного интерфейса на графический требует переписать всю бизнес-логику, разделение ответственности выполнено плохо!

6. Этапы и контрольные точки

Этап	Что проверяется	Фиксация
0. Выбор	Тема, индивидуальная проектная работа, предварительная сложность	Заявка на проект
1. Product brief	Пользователь, проблема, конкурирующий способ решения, границы MVP	docs/PRODUCT.md, тег v0.1
2. Архитектура	Модули, классы, зависимости, хранение, альтернативы	docs/ARCHITECTURE.md, тег v0.2
3. Вертикальный прототип	Один сценарий проходит через все слои от ввода до результата	Демонстрация, тест, тег v0.3
4. Рабочее ядро	Основные сценарии, ошибки, сохранение, тесты	Версия v0.5
5. MVP	Удобный запуск, документация, демонстрационные данные	Версия v0.9
6. Защита	Демонстрация, вопросы, изменение кода на месте	Релиз v1.0

Конкретные даты этапов преподаватель устанавливает в календаре дисциплины. Пропущенные контрольные точки не заменяются одним итоговым показом в конце семестра без отдельного согласования с преподавателем.

7. Правила использования ИИ

7.1. Разрешено и рекомендуется

обсуждать требования и задавать уточняющие вопросы;
просить несколько архитектурных вариантов и сравнивать их;
генерировать ограниченный, проверяемый фрагмент кода;
просить объяснить код построчно или на уровне объектов и зависимостей;
генерировать тестовые случаи и искать непокрытые границы;
проводить ревью и искать дефекты;

создавать черновики документации с последующей проверкой студентом.

7.2. Недопустимо

получить проект целиком одним запросом и представить без поэтапной разработки;

включить код, назначение которого студент не может объяснить;

скрывать факт использования ИИ или выдумывать журнал работы;

принимать ответ модели без компиляции, тестирования и анализа;

просить ИИ имитировать историю Git задним числом;

передавать нейросети реальные секреты, пароли, токены и персональные данные.

Критерий осознанности: студент должен уметь объяснить, что именно предложил ИИ, как это было проверено, что было изменено и почему.

8. Журнал работы с ИИ

Полная переписка с нейросетью не требуется. В AI_LOG.md фиксируются только существенные эпизоды, повлиявшие на архитектуру, реализацию, тесты или исправление ошибки.

Поле	Содержание
Дата/задача	Что требовалось решить
Контекст и запрос	Краткий промпт или ссылка на сохранённый фрагмент
Предложение ИИ	Суть ответа без многословного копирования
Проверка	Сборка, тест, эксперимент, документация, сравнение вариантов
Решение студента	Принято, изменено или отвергнуто; почему
Фиксация	Хеш или ссылка на коммит

9. Требования к Git

репозиторий создаётся в начале проекта, а не перед защитой;

один коммит отражает одно законченное логическое изменение;

сообщение описывает результат: «Добавлена проверка формата даты», а не «работа»;

нерабочие эксперименты ведутся в отдельной ветке или исправляются до контрольной версии;

секреты, каталоги сборки и временные файлы исключаются через .gitignore;

контрольные точки отмечаются тегами v0.1, v0.2, v0.3, v0.5, v0.9, v1.0;

CHANGELOG.md содержит изменения каждой предъявляемой версии.

Примеры сообщений коммитов

Хорошо	Плохо
Добавлен импорт транзакций из CSV	код
Исправлено двойное сохранение заказа	fix
Выделен интерфейс IReportExporter	переделал классы
Добавлены тесты пограничных дат	тесты

10. Структура репозитория

```
project/
├── CMakeLists.txt
├── README.md
├── CHANGELOG.md
├── AI_LOG.md
├── docs/
│   ├── PRODUCT.md
```



```
│   ├── ARCHITECTURE.md
│   └── DECISIONS.md
├── include/
├── src/
├── tests/
└── data/
```

Допускается другая структура, если она соответствует инструментам проекта и логически объяснена.

11. Продуктовая часть

Студент выступает не только как программист, но и как автор продукта. От него не требуется открыть бизнес или получить реальные продажи, однако необходимо проверить, существует ли понятная ценность для пользователя.

кто конкретно является пользователем;

какую регулярную проблему решает продукт;

как пользователь решает её сейчас;

почему новое решение удобнее, быстрее или безопаснее;

что входит в MVP и что сознательно отложено;

какая модель распространения возможна: бесплатно, разовая покупка, подписка, корпоративная лицензия;

какие два-три аналога или заменителя существуют;

какая предполагаемая цена и почему пользователь может согласиться её заплатить.

Важно: эффектный интерфейс и упоминание ИИ сами по себе не создают ценность. Защищается конкретная проблема пользователя и работающий способ её решения.

Защищается конкретная проблема пользователя и работающий способ ее решения.

12. Оценивание

Компонент	Вес	Что оценивается
Работоспособность и завершённость	20%	Основной сценарий, сборка, устойчивость, демонстрация
ООП и предметная модель	20%	Инкапсуляция, обязанности классов, композиция, оправданный полиморфизм
Архитектура и модульность	15%	Разделение слоёв, зависимости, расширяемость
Понимание и защита	20%	Объяснение кода, решений и способность самостоятельного изменения
Git и фиксация	10%	История, коммиты, теги, CHANGELOG, решения
Тестирование и ошибки	10%	Тесты, границы, обработка некорректных данных
Продуктовое мышление	5%	Пользователь, проблема, аналоги, упаковка, цена

Ограничитель оценки: если студент не понимает код или не способен внести небольшое изменение, проект не получает положительную итоговую оценку независимо от внешней эффектности.

13. Формат итоговой защиты

Краткий продуктовый питч: пользователь, проблема, решение (до 2 минут).

Демонстрация основного и одного ошибочного сценария (до 5 минут).

Объяснение архитектуры и зависимостей (до 5 минут).

Показ истории Git и одного существенного эпизода работы с ИИ (до 3 минут).

Ответы по ООП, коду, тестам и принятым решениям.

Небольшое изменение проекта на месте или точный план изменения с реализацией ключевого фрагмента.

Примеры вопросов

Почему этот класс существует отдельно?

Какая ответственность у данного модуля и чего он делать не должен?

Где используется полиморфизм и какую проблему он решает?

Почему выбрана композиция, наследование или шаблон?

Как заменить хранилище, не переписывая предметную логику?

Какую ошибку допустил ИИ и как она была обнаружена?

Как восстановить последнюю стабильную версию?

Что было исключено из MVP и почему?

14. Индивидуальная и командная работа

Базовый формат — индивидуальный проект.

15. Каталог рекомендуемых тем

Тема может быть выбрана из каталога или предложена студентом. Собственная тема должна иметь конкретного пользователя, содержательную предметную модель и реалистичный MVP.

№	Тема	Уровень	MVP	Ключевые объекты
1	Smart File Organizer	Базовый/средний	Правила сортировки файлов, план операций, история и откат	SortingRule, Condition, Action, FileScanner, OperationPlan
2	Personal Expense Analyzer	Средний	Импорт выписки, категории, бюджет и отчёты	Transaction, Importer, CategorizationRule, Budget, Report
3	Duplicate File Inspector	Средний	Безопасный поиск дубликатов и карантин	ComparisonStrategy, DuplicateGroup, CleanupPlan, Quarantine
4	Teacher Assignment Manager	Базовый/средний	Группы, задания, сдачи, оценки и ведомость	Student, Group, Assignment, Submission, Grade, Report
5	C++ Structure Analyzer	Продвинутый	Метрики учебного проекта и архитектурные предупреждения	SourceFile, ClassInfo, Metric, AnalysisRule, Finding
			Снимки версии файлов, восстановление	Snapshot, FileVersion, BackupEngine

6	Backup Version Manager	Продвинутый	Снапшоты, версии файлов, восстановление и политика хранения	Snapshot, FileVersion, BackupEngine, RestorePlan, RetentionPolicy
7	Desktop Time Tracker	Базовый/средний	Проекты, задачи, интервалы времени и отчёты	Project, Task, TimeSession, Timer, ReportStrategy
8	Equipment Maintenance Manager	Средний	Учёт оборудования, обслуживания и сроков	Equipment, MaintenanceEvent, Schedule, ServiceRule
9	Personal Knowledge Organizer	Средний	Заметки, теги, связи, поиск и экспорт	Note, Tag, NoteLink, SearchStrategy, Exporter
10	Test Generator and Trainer	Базовый/средний	Разные типы вопросов, прохождение и анализ ошибок	Question hierarchy, TestSession, Evaluator, ResultReport
11	Small Business Order Manager	Средний	Заказы для конкретной ниши: мастер, 3D-печать, репетитор	Customer, Order, Product, Service, Payment, Status
12	Estimate Builder	Базовый/средний	Каталог, смета, скидки, наценки и экспорт	CatalogItem, Estimate, PricingRule, DocumentExporter
13	Game Save Manager	Средний	Профили игр, резервные копии и конфликты сохранений	Game, SaveSnapshot, BackupStrategy, ConflictResolver
14	Smart Log Analyzer	Средний/продвинутый	Разбор форматов логов, фильтры, инциденты и отчёты	LogParser, Filter, AnalysisRule, Incident, Report
15	Local Password Vault	Продвинутый	Учебное локальное хранилище секретных записей	Vault, Entry hierarchy, EncryptionService, Session, Storage

Примечание к теме 15: учебный проект нельзя рекламировать как доказанно безопасный коммерческий менеджер паролей без профессионального аудита криптографии и реализации.

1. Smart File Organizer

Продукт: приложение для автоматической сортировки файлов.
Пользователь задаёт правила: документы перемещать в одну папку, изображения — в

другую, старые архивы — отдельно. Программа сначала показывает план операций, а затем выполняет подтверждённые действия.
Классы:

- `FileInfo`;
- `FileScanner`;
- `SortingRule`;
- `RuleCondition`;
- `RuleAction`;
- `OrganizationPlan`;
- `OperationHistory`;
- `ConfigRepository`.

Полиморфизм удобно реализовать через разные условия и действия.
Коммерческий смысл: утилита для наведения порядка в папках Downloads и Desktop.
Расширение: ИИ (через API) предлагает правила сортировки по именам файлов.

2. Personal Expense Analyzer

Продукт: локальный анализатор личных расходов.
Программа импортирует CSV-выписку, распознаёт категории операций, строит месячную статистику и выявляет необычные расходы.
Классы:

- `Transaction`;
- `Account`;
- `Category`;
- `CategorizationRule`;
- `StatementImporter`;
- `Budget`;
- `Report`;
- `Storage`.

Коммерческий смысл: приватный офлайн-анализ финансов без передачи банковских данных внешним сервисам.
Расширение: локальная или внешняя модель помогает классифицировать непонятные операции.

3. Duplicate File Inspector

Продукт: безопасный поиск дубликатов файлов.
Программа ищет одинаковые файлы, группирует их, показывает причины совпадения и предлагает варианты очистки. Удаление должно быть обратимым либо заменено перемещением в карантин.
Классы:

- `FileEntry`;
- `FileScanner`;
- `HashCalculator`;
- `DuplicateGroup`;
- `ComparisonStrategy`;
- `CleanupPlan`;
- `Quarantine`;

- `ScanReport`.

Коммерческий смысл: понятная утилита очистки диска.
Важный момент: стратегия сравнения может быть отдельной иерархией классов.

4. Teacher Assignment Manager

Продукт: локальный менеджер заданий и сдач для преподавателя.
Преподаватель создаёт дисциплины, группы, задания, отмечает сдачу и формирует ведомость.
Классы:

- `Student`;

- `Group`;
- `Course`;
- `Assignment`;
- `Submission`;
- `Grade`;
- `DeadlinePolicy`;
- `ReportGenerator`.

Коммерческий смысл: лёгкая альтернатива перегруженным LMS.
Расширение: ИИ (через API) формирует вопросы для защиты по описанию работы.

5. Source Code Structure Analyzer

Продукт: анализатор структуры учебных C++-проектов.
Программа сканирует исходники и составляет отчёт:

- количество файлов и классов;
- размер функций;
- наличие слишком больших классов;
- повторяющиеся имена;
- подозрительно длинные методы;
- зависимости между файлами.

Это не полноценный компилятор и не статический анализатор уровня Clang. Достаточно честно ограниченного анализатора учебных проектов.
Классы:

- `SourceFile`;
- `ClassInfo`;
- `FunctionInfo`;
- `ProjectScanner`;
- `Metric`;
- `AnalysisRule`;
- `Finding`;
- `ReportFormatter`.

Коммерческий смысл: образовательный инструмент для студентов и преподавателей.
Расширение: ИИ (через API) объясняет обнаруженные архитектурные проблемы.

10 из 17

6. Backup Version Manager

Продукт: простое резервное копирование с версиями.
Пользователь создаёт профиль резервного копирования, выбирает папки, запускает создание снимка и может восстановить определённую версию файла.
Классы:

- `BackupProfile`;
- `BackupSource`;
- `Snapshot`;
- `FileVersion`;
- `BackupEngine`;
- `RestorePlan`;
- `RetentionPolicy`;
- `ManifestRepository`.

Коммерческий смысл: понятный инструмент локального версионирования документов.
Комментарий: тема естественно связана с идеей фиксации работающих состояний.

7. Desktop Time Tracker

Продукт: локальный трекер времени по задачам.
Пользователь создаёт проекты и задачи, запускает таймер, затем получает отчёты.
Классы:

- `Project`;
- `Task`;
- `TimeSession`;
- `Timer`;
- `Tag`;
- `Report`;
- `ReportStrategy`;
- `DataRepository`.

11 из 17

Коммерческий смысл: простой офлайн-трекер без подписки и регистрации.
Расширение: ИИ (через API) составляет краткий отчёт о рабочем дне.

8. Equipment Maintenance Manager

Продукт: учёт обслуживания автомобилей, компьютеров, принтеров или лабораторного оборудования.
Классы:

- `Equipment`;
- `MaintenanceEvent`;
- `MaintenanceSchedule`;
- `Notification`;
- `ServiceRule`;
- `Document`;
- `Repository`;
- `ReportGenerator`.

11 из 17

Можно создать наследников `Vehicle`, `Computer`, `Printer`, но студент должен объяснить, почему наследование здесь оправдано.
Коммерческий смысл: продукт для маленькой мастерской, кафедры или владельца нескольких автомобилей.

9. Local Password Vault — учебная версия

Продукт: локальное хранилище секретных записей.
Классы:

- `Vault`;
- `VaultEntry`;
- `CredentialEntry`;
- `SecureNote`;
- `EncryptionService`;
- `PasswordGenerator`;
- `Storage`;
- `Session`.

Коммерческий смысл: приватная локальная утилита.
Ограничение: проект нельзя представлять как реально безопасный коммерческий менеджер паролей без профессионального аудита. Это хорошая тема для разговора о границах компетенции и ответственности разработчика.

10. Personal Knowledge Organizer

Продукт: локальная база заметок с тегами, связями и поиском.
Классы:

- `Note`;
- `Tag`;
- `Notebook`;
- `NoteLink`;
- `SearchQuery`;
- `SearchStrategy`;
- `Repository`;
- `Exporter`.

Коммерческий смысл: компактная офлайн-альтернатива крупным системам заметок.
Расширение: ИИ (через API) создаёт краткое содержание заметки или предлагает теги.

11. Test Generator and Trainer

Продукт: редактор тестов и режим тренировки.
Есть разные виды вопросов, случайная генерация варианта, прохождение теста и анализ ошибок.
Классы:

- `Question`;
- `SingleChoiceQuestion`;
- `MultipleChoiceQuestion`;
- `TextQuestion`;

- `Test`;
- `TestSession`;
- `AnswerEvaluator`;
- `ResultReport`.

Коммерческий смысл: преподаватель может создавать локальные тренажёры.
Сильная сторона: естественный полиморфизм разных типов вопросов.

12. Small Business Order Manager

Продукт: система учёта заказов для мастера, самозанятого или небольшой мастерской.
Классы:

- `Customer`;
- `Order`;
- `OrderItem`;
- `Product`;
- `Service`;
- `Payment`;
- `OrderStatus`;
- `InvoiceGenerator`.

Коммерческий смысл: конкретный продукт для репетитора, ремонтника, 3D-печатника, фотографа или кондитера.
Студент должен выбрать одну реальную нишу, поговорить хотя бы с одним потенциальным пользователем и адаптировать приложение под его работу.

13. Price List and Estimate Builder

Продукт: создание смет и коммерческих предложений.
Классы:

- `Catalog`;
- `CatalogItem`;
- `Material`;
- `Service`;
- `Estimate`;
- `EstimateItem`;
- `PricingRule`;
- `DocumentExporter`.

Коммерческий смысл: программа для небольших ремонтных бригад, мастерских и частных специалистов.
Расширение: разные стратегии расчёта скидок, налогов и наценок.

14. Game Save Manager

Продукт: резервное копирование и переключение сохранений игр.
Классы:

- `Game`;
- `SaveLocation`;

- `SaveSnapshot`;
- `GameProfile`;
- `BackupStrategy`;
- `RestoreOperation`;
- `ConflictResolver`;
- `Storage`.

Коммерческий смысл: нишевая утилита для игроков.
Плюс: тема выглядит живее традиционного «учёта книг», но содержит полноценную архитектурную задачу.

15. Smart Log Analyzer

Продукт: анализатор журналов программ и серверов.
Программа читает логи разных форматов, фильтрует события, группирует ошибки и формирует отчёт.
Классы:

- `LogEntry`;
- `LogParser`;
- `LogFormat`;
- `Filter`;
- `AnalysisRule`;
- `Incident`;
- `Statistics`;
- `ReportExporter`.

Коммерческий смысл: небольшая диагностическая утилита.
Расширение: ИИ (через API) переводит техническую группу ошибок в понятное объяснение.

16. Уровни сложности

Уровень	Ориентиры	Комментарий
Средний	Несколько сценариев, интерфейсы, сменные стратегии, тесты, JSON/SQLite, продуманная обработка ошибок	
Продвинутый	Системные API, конкурентность, GUI, плагины, сложное хранение или встроенный ИИ	Требует раннего прототипа и управления рисками

Хорошо завершённый средний проект ценнее недоделанного продвинутого.

17. Заявка на собственную тему

рабочее название;

один конкретный тип пользователя;

описание проблемы в 3–5 предложениях;

основной пользовательский сценарий;

предварительные сущности и классы;

границы MVP;

два аналога или существующих способа решения;

основные технические риски;

план первой вертикальной версии.

Приложение А. Шаблон PRODUCT.md

Название: Краткое и рабочее; может измениться.
Пользователь: Один основной сегмент, а не «все люди».
Проблема: Как проявляется и как часто возникает.
Текущий способ: Чем пользователь обходится сегодня.
Ценность: Что становится быстрее, дешевле, удобнее или безопаснее.
MVP: Минимальный сквозной набор функций.
Не входит: Сознательно отложенные функции.
Аналоги: Прямые конкуренты и заменители.
Монетизация: Предполагаемая цена/модель и аргумент.
Критерий готовности: Наблюдаемый результат, после которого MVP считается законченным.

Приложение Б. Шаблон архитектурного решения

Дата и статус: предложено / принято / заменено.

Контекст: какая проблема или ограничение возникло.

Рассмотренные варианты: минимум два, если решение существенно.

Решение: что именно выбрано.

Причины: почему выбран этот вариант.

Последствия: преимущества, ограничения и будущие работы.

Связанные коммиты и версии.

Приложение В. Шаблон записи AI_LOG.md

```
## 2026-__-__ — краткое название задачи

**Контекст:** ...

**Запрос к ИИ:** ...

**Предложение ИИ:** ...

**Как проверено:** ...

**Моё решение:** принято / изменено / отвергнуто, потому что ...

**Коммит:** ...
```


Приложение Г. Чек-лист готовности к защите

[ ] Репозиторий собирается по инструкции на чистом каталоге сборки.

[ ] Основной сценарий работает на демонстрационных данных.

[ ] Показан хотя бы один ошибочный или пограничный сценарий.

[ ] Тесты запускаются одной командой и завершаются успешно.

[ ] README, CHANGELOG, AI_LOG и архитектурные документы актуальны.

[ ] Контрольные версии отмечены тегами.

[ ] В репозитории нет паролей, токенов и каталогов сборки.

[ ] Студент может нарисовать основные зависимости без подсказки ИИ.

[ ] Подготовлено объяснение одного ошибочного ответа ИИ.

[ ] Подготовлена небольшая идея расширения и указано место её реализации.

Приложение Д. Решение преподавателя

Поле	Заполняется
Студент / группа	
Тема	
Формат	Индивидуально
Уровень	Базовый + средний / продвинутый
Особые обязательные функции	
Согласованные контрольные даты	
Решение	Одобрено / требуется доработка / отклонено
Комментарий преподавателя	

Москва · 2026